

SYSTEM ANALYST

presented by Ayub

SQL Basics

Данные — это новое золото.

Компании, у которых лучше данные, побеждают конкурентов.

Базы данных — это бизнес.

Netflix: без БД vs с БД

Вопрос: Как Netflix знает какой фильм тебе понравится? 🖼️🤔

(Жди ответы студентов: "алгоритм", "AI", "какие-то данные")

Ответ: Алгоритм работает с **данными**. Откуда они? Из базы данных твоей истории просмотров.

- **Без БД:** ты видишь список всех фильмов. Скучно. Закрываешь.
- **С БД:** Netflix помнит что ты смотрел. Знает твой вкус. Рекомендует точно твоё.

Без БД → откуда алгоритму взять информацию для рекомендации?

Netflix: 80% просмотров = рекомендации

- 80% всех просмотров идут из рекомендаций (не из поиска)
- Рекомендации = персонализация = люди остаются в Netflix

Без базы данных → нет рекомендаций → потеря 80% доходов

Spotify: память о каждой песне

- Spotify помнит **каждую песню** которую ты слушал
- Вся эта история = в базе данных
- Алгоритм находит людей похожих на тебя (по вкусу)
- Рекомендует их песни тебе

Discover Weekly = БД + алгоритм

- Ты открываешь Discover Weekly → находишь новую любимую песню → остаёшься в Spotify
- Если закроешь БД → исчезнет вся история → Spotify станет бесполезен

История без БД = рекомендации исчезают = пользователь уходит

Amazon: от книг к миру

- Amazon начал как интернет-**книжный магазин** (1995)
- Теперь → всё: одежда, еда, электроника, облако, видео
- Как они расширялись быстрее конкурентов?

Ответ: данные клиентов. Знаешь что люди хотят → продашь им всё.

Amazon: "Люди также покупали..."

- Amazon помнит **каждый поиск, клик, покупку**
- "Люди также покупали..." → из БД твоих покупок
- Рекомендации = 20% дохода Amazon
- Рекомендации дают Amazon силу расширяться на новые категории

Другие магазины гадают. Amazon знает. Вот почему Amazon побеждает.

Uber: как работает приложение без базы данных?

Вопрос: Как Uber находит ближайшего водителя за 5 секунд? 📄 🚗

Представь Uber без БД:

- Водитель звонит: "Я свободен"
- Пассажир звонит: "Мне нужна машина"
- Диспетчер соединяет по телефону
- Водитель едет случайно (не знает где пассажир)
- Платёж — наличные (нет истории)

Это же — оффлайн такси (1990е годы).

Вопрос: Можно ли найти ближайшего водителя БЕЗ базы данных?

Но Uber — это базы данных:

- БД знает где водитель (GPS)
- БД знает где пассажир (GPS)

SQL • БД находит ближайшего водителя за 5 секунд

- БД обрабатывает платёж онлайн

Airbnb: доверие без компании

Оффлайн:

- Звоню хозяину "Вы честный человек?"
- Он: "Да конечно!"
- Ты веришь? Нет.

Airbnb (с БД):

- Я вижу 150 отзывов от других гостей
- Рейтинг 4.9/5
- "Квартира идеальна!", "Хозяйка отзывчивая"
- Теперь я доверяю. Бронирую.

БД = система доверия

Отзывы в БД = репутация. Без БД отзывов = нет доверия = нет бизнеса Airbnb.

- Люди верят не Airbnb, а **систему отзывов в БД**
- Хосты доверяют гостям через рейтинги
- Гости доверяют хостам через отзывы

Быстрая проверка: компании и данные

Твоё понимание:

1. Какая компания из примеров **первой** начала использовать данные для рекомендаций — Netflix, Spotify или Amazon?
2. Почему Uber **не может** работать без базы данных? (Напиши 1-2 причины)
3. Какой процент просмотров на Netflix идёт из рекомендаций? *(Подсказка: огромный)*

(3-4 минуты на обсуждение)

А теперь обратное: что если БД потеряется?

Или если её украдут?

Или если её удалят?

Equifax 2017: 147 миллионов человек

Equifax = кредитное агентство (хранит финансовую историю).

Хакеры влезли и украли:

- Имена (147M)
- Номера социального страхования (147M) ← это паспортный номер в США
- Даты рождения (147M)
- Водительские права (147M)

Теперь какой-то преступник где-то может:

- Открыть кредит под твоё имя
- Получить кредитную карту
- Купить машину в твой кредит
- Твой рейтинг кредита разрушен

Люди: "Мы требуем сильной защиты БД!?"

Equifax: "Ладно, заплатим \$700 миллионов в штрафах и судах"

Банк теряет БД = Страна теряет экономику

Представь:

- Завтра банк теряет все данные
- Никаких записей о счётах
- Никаких записей о кредитах
- Никаких записей о зарплатах

Ты приходишь в банк:

- "У меня там \$50,000"
- Банк: "В БД не видим. Извини, забыли."
- Ты: "Я куплю это в кредит?"
- Банк: "Какой кредит? Нет данных что ты честный платильщик."

Результат:

- Никто не верит банкам
- Никто не открывает счёты
- Бизнес не может брать кредиты
- Экономика замораживается

Хакеры целются в данные. Не в деньги.

Старый стереотип: "Хакер взламывает банк и крадёт деньги"

Реальность: "Хакер крадёт БД клиентов и продаёт"

Почему?

- Украсть \$1M из банка = уходишь в тюрьму навсегда
- Украсть БД 100M клиентов = продаешь на тёмном рынке за \$5M и сидишь в другой стране

Что хакер может сделать с твоей БД:

- Продать список email + пароли → спам
- Продать номера кредитных карт → кража денег
- Продать личные данные → идентификационное мошенничество
- Удалить всё чтобы вымогать выкуп: "Верни БД или платишь \$1M"

Твоя работа = защитить БД от хакеров. Если не защитишь → компания теряет миллионы + репутация разрушена.

Instagram: от 100K к 10M

2010 (100K пользователей):

- "Сохраняем все фото в одной таблице"
- Работает. Быстро.

2012 (10M пользователей):

- Фото грузятся 10 секунд
- БД медленная
- **Нужно переделать архитектуру**
- Инженеры работают неделями
- Стоимость: миллионы долларов + downtime

Переделать дизайн = миллионы долларов

Twitter: Дизайн не масштабировался → переделали всю систему → годы работы, миллионы

YouTube: Видео хранили неправильно → мигрировали на распределённую систему → downtime

Вывод:

Правильный дизайн на старте = масштабируется без переделок. Плохой дизайн = дорогой ремонт потом.

Backup = страховка от катастрофы

Сценарий 1: Пожар в дата-центре

- Все серверы горят
- Все БД = угли
- Нет backup → данные потеряны навсегда
- Компания: "Извините, все ваши данные удалены"
- Клиенты: "Мы подаём в суд"
- Итог: компания закрывается

Сценарий 2: Ransomware атака

- Хакер: "Платите \$5M или я удалю БД"
- Компания: "Но мы делали backup!"
- Стирает заражённую БД, восстанавливает backup
- Потеря данных за 1 день (последний backup)
- Но компания выжила

Правило 3-2-1:

- **3 копии** данных (оригинал + 2 backup)

Ты проектируешь будущее

Вопрос: Ты думаешь SQL это просто "составить запрос и получить данные"?

Реальность:

- Проектировать БД, которая хранит **миллиарды** записей
- Защищать БД от хакеров (Equifax потеряла 147M записей)
- Создавать backup, чтобы данные не потерялись (потеря = закрытие компании)
- Оптимизировать запросы (медленный запрос → уходят клиенты)

Если ты ошибёшься:

- Клиенты потеряют доверие
- Компания потеряет миллионы
- Люди потеряют работу

Твоя работа = ответственность. Большая ответственность.

(Это не пугалка — это реальность инженера)

Почему мы учимся SQL

Вопрос: Если Netflix нужно выбрать из своей БД 10 фильмов специально для тебя — как это происходит?

(Жди ответы: "программист пишет код", "SQL запрос", "компьютер ищет"...)

SQL = язык для разговора с базами данных.

Почему он важен:

- Без SQL не можешь создать БД
- Без SQL не можешь читать данные
- Без SQL не можешь защитить БД
- Без SQL не можешь масштабировать систему

Шесть SQL команд, которые ты выучишь сегодня → могут управлять миллиардами записей

Сегодня ты выучишь 6 команд:

- CREATE TABLE (создать)
- INSERT INTO (добавить)
- SELECT (читать)

Зачем вообще базы данных?

Три вопроса на размышление:

- Инстаграм хранит **миллиарды** фото — где? 📷
- Банк помнит **каждый** твой платёж — как? 💳
- Spotify знает твою историю прослушиваний — в чём? 🎵

(Жди ответы студентов: "сервер", "облако", "БД", "компьютер"...)

Всё это — базы данных. SQL — язык, на котором с ними разговаривают.

Что мы изучим сегодня

Вопрос: Какими инструментами инженер управляет базой данных компании?

Ответ: Ровно этими 6 командами:

- **CREATE TABLE** — создать таблицу (структура)
- **INSERT INTO** — добавить записи (данные)
- **SELECT** — получить данные (читать)
- **UPDATE** — изменить записи (обновить)
- **DELETE** — удалить записи (удалить)
- **ALTER TABLE** — изменить структуру (адаптировать)

Плюс:

- Типы данных (`INT` , `VARCHAR` , `DECIMAL` , `BOOLEAN` , `DATE`)
- `WHERE` фильтрация (выбирать конкретные строки)
- `AND` логика (комбинировать условия)

Всё остальное в SQL строится на этих базах. Овладей ими — и ты сможешь работать с любой БД.

Вы уже рисовали Class Diagram — класс `User` с атрибутами `name`, `email`.

А как хранить тысячи таких пользователей в реальной системе?

(Жди ответы: "список", "файл", "массив", "Excel", "БД"...)

Как это выглядит в коде?

Таблица = Класс. Строка = Объект.

- **Таблица (Table)** — структура хранения, как класс в UML
- **Столбец (Column)** — атрибут класса (`name` , `email`)
- **Строка (Row)** — конкретный объект / экземпляр (`Алексей, alex@mail.ru`)
- **Ячейка (Cell)** — одно конкретное значение (`"Алексей"`)

Пример: Class Diagram vs SQL Таблица

id	name	email	age
1	Алексей	alex@mail.ru	28
2	Мария	maria@gmail.com	24
3	Данияр	daniyar@yandex.kz	31

- Класс `User` описывает **что хранить**
- Таблица `users` **хранит** реальные данные
- Каждая строка — отдельный человек
- Первичный ключ **(Primary Key)** `id` — уникальный номер каждой строки

Module Check — Таблицы

1. **Чем таблица похожа** на Class Diagram?
2. **Что такое строка** в базе данных — своими словами?
3. **Зачем нужен** столбец `id` ?

У нас есть поле `price` для цены товара.

Какой тип лучше: `INT` или `DECIMAL` ? Почему?

Типы данных в SQL

Тип	Что хранит	Пример
INT	Целое число	ID: 42 , возраст: 28
VARCHAR(n)	Строка до n символов	Имя: 'Алексей'
TEXT	Длинный текст (без лимита)	Комментарий, описание
BOOLEAN	Истина / ложь	Активен: true
DATE	Дата	Дата рождения: 2000-03-15
DECIMAL(p, s)	Дробное число	Цена: 10.99
TIMESTAMP	Дата + время	Создан: 2024-01-15 14:30:00

VARCHAR vs TEXT — когда что использовать?

- `VARCHAR(n)` — строка с ограничением длины
 - Имя: `VARCHAR(100)` — имя не бывает длиннее 100 символов
 - Email: `VARCHAR(255)` — стандартный максимум
 - Используй когда знаешь максимальную длину
- `TEXT` — строка без ограничения
 - Комментарий пользователя
 - Описание товара
 - Биография профиля

Правило: знаешь максимум → `VARCHAR` . Не знаешь → `TEXT` .

Module Check — Типы данных

1. **Какой тип** выбрать для поля `phone_number` ?
2. **Чем** `DECIMAL` лучше `INT` для цены?
3. **Когда** использовать `TEXT` вместо `VARCHAR` ?
4. **Какой тип** для поля "дата регистрации пользователя"?

Мы знаем какие поля нужны и какие у них типы.

Как теперь создать саму таблицу?

CREATE TABLE — синтаксис

```
CREATE TABLE имя_таблицы (  
    имя_столбца ТИП_ДАННЫХ,  
    имя_столбца ТИП_ДАННЫХ,  
    ...  
);
```

- `CREATE TABLE` — команда создания
- Внутри скобок — список столбцов
- Каждый столбец: имя + тип через пробел
- Столбцы разделяются запятой
- В конце — `;`

CREATE TABLE users

```
CREATE TABLE users (  
  id          INT,  
  name        VARCHAR(100),  
  email       VARCHAR(255),  
  age         INT,  
  is_active   BOOLEAN,  
  created_at  DATE  
);
```

- Таблица `users` — основа всего урока
- 6 столбцов с разными типами
- `is_active` — активен ли аккаунт
- `created_at` — дата регистрации

CREATE TABLE orders

```
CREATE TABLE orders (  
  id          INT,  
  user_id     INT,  
  product_name VARCHAR(200),  
  price       DECIMAL(10, 2),  
  order_date  DATE  
);
```

- `user_id` — ссылка на пользователя из `users`
- `DECIMAL(10, 2)` — число с двумя знаками после запятой
- Эти две таблицы будем использовать весь урок

Module Check — CREATE TABLE

1. **В чём разница** между `CREATE TABLE` и классом из UML?
2. **Какую ошибку** здесь? (укажи строку)

```
CREATE TABLE users (  
    id INT,  
    name VARCHAR(100),  
);
```

3. **Почему** мы используем `DECIMAL` для цены, а не `INT` ?
4. **Если я забуду указать тип** для столбца — что произойдёт?

(4-5 минут на обсуждение, потом сразу практика)

Практика 1 — CREATE TABLE

Задача: создай таблицу `products`

Поле	Тип
<code>id</code>	INT
<code>name</code>	VARCHAR(200)
<code>price</code>	DECIMAL(10, 2)
<code>in_stock</code>	BOOLEAN
<code>added_date</code>	DATE

(5-7 минут самостоятельной работы)

Практика 1 — шаги

1. Напиши `CREATE TABLE products (`
2. Добавь каждое поле с типом через запятую
3. Закрой скобку и поставь `;`

```
CREATE TABLE products (  
    id          INT,  
    name        VARCHAR(200),  
    price        DECIMAL(10, 2),  
    in_stock     BOOLEAN,  
    added_date   DATE  
);
```

Таблица создана. Она пустая.

Как добавить туда первые данные?

(Жди ответы: "написать данные", "заполнить", "SQL-команда"...)

Вопрос на размышление: *Если я добавлю данные в неправильном порядке столбцов — что произойдёт?*

INSERT INTO — два варианта

Вариант 1 — без имён столбцов (опасно):

```
INSERT INTO users VALUES (1, 'Алексей', 'alex@mail.ru', 28, true, '2024-01-10');
```

Вариант 2 — с именами столбцов (безопасно):

```
INSERT INTO users (id, name, email, age, is_active, created_at)  
VALUES (1, 'Алексей', 'alex@mail.ru', 28, true, '2024-01-10');
```

Всегда используй вариант 2 — если порядок столбцов изменится, данные не перепутаются.

Заполняем таблицу users

```
INSERT INTO users (id, name, email, age, is_active, created_at) VALUES
(1, 'Алексей', 'alex@mail.ru', 28, true, '2024-01-10'),
(2, 'Мария', 'maria@gmail.com', 24, true, '2024-02-15'),
(3, 'Данияр', 'daniyar@mail.ru', 31, false, '2023-11-01'),
(4, 'Айгерим', 'aigul@yandex.kz', 19, true, '2024-03-20'),
(5, 'Павел', 'pavel@inbox.ru', 65, false, '2022-07-08');
```

- 5 пользователей с разными данными
- `is_active = false` у некоторых — пригодится для фильтрации

Заполняем таблицу orders

```
INSERT INTO orders (id, user_id, product_name, price, order_date) VALUES
(1, 1, 'Наушники Sony', 3500.00, '2024-03-01'),
(2, 1, 'Ноутбук Lenovo', 85000.00, '2024-03-15'),
(3, 2, 'Мышь Logitech', 990.00, '2024-04-02'),
(4, 3, 'Клавиатура', 2200.00, '2024-04-10'),
(5, 4, 'Кабель USB', 150.00, '2024-04-12');
```

- `user_id` указывает кто сделал заказ
- Разные цены — пригодится для фильтрации по `WHERE`

Практика 2 — INSERT INTO

Задача: вставь 3 записи в таблицу `products`

Шаги:

1. Напиши `INSERT INTO products (name, price, in_stock, added_date)`
2. Добавь `VALUES` и первую строку данных
3. Повтори для 3 строк (можно в одном INSERT через запятую)

Примеры данных:

- Телефон Samsung, 45000.00, true, 2024-01-05
- Чехол для телефона, 800.00, true, 2024-01-10
- Зарядное устройство, 1200.00, false, 2023-12-20

Module Check — INSERT INTO

1. **Чем опасен** `INSERT INTO users VALUES (...)` без указания названий столбцов?
2. **Как выглядит** INSERT с правильными названиями столбцов? (напиши первую строку)
3. **Если я вставлю дату в формате** `01.03.2024` вместо `2024-03-01` — что произойдёт?
4. **Можно ли вставить пустое значение** (NULL) в столбец `id` ?

(4-5 минут на обсуждение)

У нас 1000 пользователей в таблице.

Как найти только тех, кто старше 25 лет?

(Жди ответы: "поиск", "фильтр", "WHERE", "условие"...)

Как это записать на SQL?

SELECT — выбрать всё

```
SELECT * FROM users;
```

- `SELECT` — что выбрать
- `*` — все столбцы
- `FROM users` — из какой таблицы

Результат: все строки, все столбцы таблицы `users`

SELECT — конкретные столбцы

```
SELECT name, email FROM users;
```

Результат:

name	email
Алексей	alex@mail.ru
Мария	maria@gmail.com
Данияр	daniyar@mail.ru

- Получаем только нужные столбцы
- Быстрее и экономичнее чем `SELECT *`

SELECT WHERE — фильтрация

```
SELECT * FROM users WHERE is_active = true;
```

```
SELECT * FROM orders WHERE price > 500;
```

```
SELECT * FROM users WHERE age ≥ 25 AND is_active = true;
```

- WHERE — условие фильтрации
- =, >, <, ≥, ≤ — операторы сравнения
- AND — оба условия должны быть истинны

Практика 3 — SELECT

Три мини-задачи:

1. Выбери все товары из `products` дороже 100
2. Выбери только `name` и `price` из `products`
3. Найди все товары в наличии (`in_stock = true`) дешевле 1500

Подсказка для задачи 3:

```
SELECT ... FROM products WHERE ... AND ...;
```

Module Check — SELECT

1. **Чем** `SELECT *` отличается от `SELECT name, email` ?
2. **Что вернёт** `SELECT * FROM users WHERE age < 20` ?
3. **Напиши запрос:** найди все заказы пользователя с `user_id = 1`
4. **Что значит** `AND` в WHERE-условии?

Module Check — SELECT (расширенный)

1. **Что выбирает** `SELECT * FROM users` ?
2. **Напиши запрос:** получи имена и email всех активных пользователей
3. **Что вернёт** этот запрос?

```
SELECT * FROM orders WHERE price > 1000 AND user_id = 1;
```

4. **Вопрос:** почему лучше писать `SELECT name, email` вместо `SELECT *` в приложении?

(5-6 минут на обсуждение)

Таблица `users` уже в работе, тысячи записей.

Менеджер говорит: нужно добавить поле `phone` . Как?

(Жди ответы: "пересоздать таблицу", "добавить новый столбец", ...)

Если пересоздать таблицу → что произойдёт со всеми записями?

ALTER TABLE — добавить столбец

```
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
```

- ALTER TABLE — изменить структуру существующей таблицы
- ADD COLUMN — добавить новый столбец
- Данные в таблице **не пропадают**
- Новый столбец для старых строк будет NULL

Бизнес-требования меняются — ALTER TABLE позволяет адаптировать таблицу без потери данных.

Практика 4 — ALTER TABLE

Задача: добавь в таблицу `products` столбец `category`

Шаги:

1. Напиши `ALTER TABLE products`
2. Добавь `ADD COLUMN category VARCHAR(50);`
3. Проверь результат: `SELECT * FROM products`

Ожидаемый результат:

- Новый столбец `category` появился
- У всех существующих товаров `category = NULL`

Module Check — ALTER TABLE

1. **Что происходит** со старыми записями когда мы добавляем новый столбец с ALTER?
2. **Правда ли:** ALTER TABLE теряет все данные? (да/нет + почему)
3. **Напиши команду:** добавь столбец `description VARCHAR(500)` в таблицу `products`
4. **Когда в продакшене** нужен ALTER TABLE вместо создания новой таблицы?

(4-5 минут)

Пользователь Данияр снова активен.

Как обновить его запись в таблице, не создавая новую?

(Жди ответы: "изменить", "обновить запись", ...)

Какая команда в SQL позволяет менять существующие данные?

UPDATE — обновить запись

```
UPDATE users SET is_active = true WHERE id = 3;
```

- UPDATE — какую таблицу менять
- SET — что изменить (столбец = новое значение)
- WHERE — какую строку менять

Обновить несколько столбцов сразу:

```
UPDATE orders  
SET price = 999.99, product_name = 'Ноутбук Pro'  
WHERE id = 2;
```

UPDATE без WHERE — это катастрофа

Вопрос: Что случится если я напишу это?

```
UPDATE users SET is_active = false;
```

(Жди ответы студентов... обычно: "все обновятся", "все станут неактивными")

Правильный вариант:

```
UPDATE users SET is_active = false WHERE id = 3;
```

Правило: перед каждым UPDATE всегда пиши WHERE. Проверь два раза.

(Это реальная ошибка джуниоров в продакшене — потом восстанавливают из backup)

Практика 5 — UPDATE

Две мини-задачи:

1. Обнови цену товара с `id = 1` в `products` на `299.99`
2. Установи `is_active = false` для всех пользователей старше 60

Шаги для задачи 2:

1. Напиши `UPDATE users`
2. Добавь `SET is_active = false`
3. Добавь `WHERE age > 60`
4. Проверь: `SELECT * FROM users WHERE age > 60`

Module Check — UPDATE (перед DELETE)

1. **Напиши UPDATE:** обнови цену заказа с id=1 на 5000.00
2. **Вопрос:** зачем нужна `WHERE` в UPDATE?
3. **Что произойдёт** если написать `UPDATE users SET age = age + 1` без WHERE?
4. **Как проверить** свой UPDATE перед тем как его запустить? (подсказка: используй SELECT)

(4-5 минут)

Пользователь попросил удалить его аккаунт.

Как удалить конкретную строку из таблицы?

(Жди ответы: "удалить запись", "DELETE", "стереть"...)

Правда ли: DELETE и DROP TABLE одно и то же?

DELETE — удалить строку

```
DELETE FROM users WHERE id = 5;
```

- `DELETE FROM` — из какой таблицы удалять
- `WHERE` — условие (какие строки удалить)
- Удалённые строки восстановить **нельзя**

DELETE vs DROP TABLE:

Команда	Что делает
<code>DELETE FROM users WHERE id = 5</code>	Удаляет одну строку
<code>DELETE FROM users</code>	Удаляет все строки, таблица остаётся
<code>DROP TABLE users</code>	Удаляет всю таблицу целиком

DELETE без WHERE — тоже катастрофа

Вопрос: Что произойдёт если я напишу:

```
DELETE FROM orders;
```

(Жди ответы: "удалит всё", "все заказы исчезнут"...)

Правильный способ:

```
DELETE FROM orders WHERE id = 5;
```

Профессиональный паттерн:

1. Сначала *SELECT* (проверь что удаляешь)
2. Потом *DELETE* (только если *SELECT* правильный)
3. Потом *SELECT* снова (проверь результат)

Практика 6 — DELETE

Две задачи:

1. Удали товар с `id = 3` из `products`
2. Удали все заказы, где `price < 10`

Шаги для задачи 2:

1. Сначала проверь: `SELECT * FROM orders WHERE price < 10`
2. Убедись что удаляешь нужные строки
3. Напиши `DELETE FROM orders WHERE price < 10;`

Module Check — UPDATE и DELETE

1. **Что произойдёт** если написать `DELETE FROM users` без WHERE?
2. **Чем** `DELETE FROM users` отличается от `DROP TABLE users` ?
3. **Напиши запрос:** обнови email пользователя с id=2 на `new@mail.ru`
4. **Почему** нужно сначала писать SELECT перед DELETE?

Итог: все 6 команд

Команда	Что делает
CREATE TABLE	Создаёт новую таблицу
INSERT INTO	Добавляет строку в таблицу
SELECT	Читает данные из таблицы
ALTER TABLE	Изменяет структуру таблицы
UPDATE	Изменяет данные в строках
DELETE	Удаляет строки из таблицы

Большая практика — задача 1

Сценарий: Зарегистрировался новый пользователь Алинур, 28 лет.

Задача: добавь его в таблицу `users`

Шаги:

1. Определи следующий свободный `id` (у нас было 5 — берём 6)
2. Напиши `INSERT INTO users (id, name, email, age, is_active, created_at)`
3. Добавь `VALUES` с реальными данными
4. Придумай email и дату регистрации

Большая практика — задача 2

Сценарий: Алинур сменил email.

Задача: обнови его email в таблице

Шаги:

1. Напиши `SELECT * FROM users WHERE id = 6` — убедись что нашёл нужную строку
2. Напиши `UPDATE users SET email = 'новый@mail.ru'`
3. Добавь `WHERE id = 6`
4. Снова сделай SELECT и проверь изменение

Большая практика — задача 3

Сценарий: Менеджер хочет список активных пользователей моложе 30.

Задача: напиши запрос

Шаги:

1. Напиши `SELECT * FROM users`
2. Добавь `WHERE is_active = true`
3. Добавь `AND age < 30`

Кто должен попасть в результат?

- Алексей (28, active) — да
- Мария (24, active) — да
- Айгерим (19, active) — да

Большая практика — задача 4

Сценарий: Менеджер просит добавить статус заказа.

Шаги:

1. Добавь в таблицу `orders` столбец `status VARCHAR(20)`

```
ALTER TABLE orders ADD COLUMN status VARCHAR(20);
```

2. Установи статус `'new'` для всех заказов

```
UPDATE orders SET status = 'new';
```

(Здесь WHERE не нужен — меняем ВСЕ строки намеренно)

Большая практика — задача 5

Сценарий: Найди дорогие заказы и пометь их как VIP.

Шаги:

1. Сначала посмотри что есть:

```
SELECT * FROM orders WHERE price > 1000;
```

2. Обнови статус для этих заказов:

```
UPDATE orders SET status = 'vip' WHERE price > 1000;
```

3. Проверь результат: `SELECT * FROM orders;`

Большая практика — задача 6

Сценарий: Пользователь с id=3 попросил удалить все его заказы.

Шаги:

1. Посмотри его заказы:

```
SELECT * FROM orders WHERE user_id = 3;
```

2. Убедись что видишь только его заказы

3. Удали:

```
DELETE FROM orders WHERE user_id = 3;
```

4. Проверь: `SELECT * FROM orders WHERE user_id = 3;`

Final Check

1. **Как создать** таблицу `categories` с полями `id` (INT) и `name` (VARCHAR(100))?
2. **Какой тип данных** выбрать для поля "описание продукта" — длина заранее неизвестна?
3. **Что вернёт** `SELECT * FROM orders WHERE price BETWEEN 1000 AND 5000` ?
4. **Напиши** UPDATE который установит `is_active = false` для пользователей с `age > 70`
5. **В чём разница** между `DELETE FROM users WHERE id = 1` и `DROP TABLE users` ?

Итоги урока — быстрая проверка

Закрой записи и ответь:

1. **Назови все 6 команд** SQL которые ты выучил сегодня
2. **В чём разница** между `UPDATE` и `DELETE` ?
3. **Что произойдёт** если забыть `WHERE` в UPDATE?
4. **Почему** `DECIMAL` лучше чем `INT` для денег?
5. **Когда** используешь ALTER TABLE вместо DROP TABLE?

(2-3 минуты, потом итоги)

Итоги урока — что ты теперь можешь

Таблица = класс из UML. **Строка** = объект. **Столбец** = атрибут.

Тип данных определяет что хранить: `INT` , `VARCHAR` , `DECIMAL` , `BOOLEAN` , `DATE` , `TIMESTAMP` .

CREATE TABLE создаёт структуру. **INSERT** заполняет. **SELECT** читает. **WHERE** фильтрует.

UPDATE и **DELETE** без `WHERE` — меняют и удаляют **ВСЕ** строки. Всегда проверяй условие.

ALTER TABLE — безопасно изменяет структуру без потери данных.

SELECT → UPDATE → SELECT — профессиональный паттерн проверки перед изменениями.

(Следующий урок — JOIN. Ты научишься объединять данные из двух таблиц.)